МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

Институт технологий управления

Центр технологий 1С

ОТЧЁТ О ПРАКТИЧЕСКИХ ЗАНЯТИЯХ № 2 И № 3

по дисциплине «Проектирование и архитектура программного обеспечения»

ТЕХНИЧЕСКОЕ ОПИСАНИЕ И АРХИТЕКТУРА ПРОЕКТА

«Персональный тайм-трекер на Java»

Выполнил: студент группы ЭПБО-01-25
Хусяинов Э. Р.

Проверил: Яковлев Михаил Сергеевич

Москва — 2026

СОДЕРЖАНИЕ

1 Введение 3
2 Общее описание продукта 5
3 Выявление и анализ требований 7
4 Требования к программному обеспечению 10
5 Спецификации вариантов использования 15
6 UML-модели системы 17
7 Верификация требований и критерии приёмки 20
8 Проектные решения и допущения 23
9 Архитектура программного обеспечения и применение паттернов проектирования 24
9.1 Архитектурные принципы и уточнение платформенной среды 24
9.2 Выбор и обоснование паттернов проектирования 25
9.3 Применение паттернов State, Observer, Strategy, Adapter и Factory Method 26
10 Система сборки и организация Java-проекта 29
10.1 Выбор Maven и сравнение систем сборки 29
10.2 Координаты и конфигурация pom.xml 30
10.3 Зависимости, структура и проверка сборки 30
Список использованных источников 31
Заключение 32
Приложение А. Протокол исходного интервью 33

1 ВВЕДЕНИЕ

1.1 Назначение документа

Настоящая спецификация требований к программному обеспечению (Software Requirements Specification, SRS) подготовлена для учебного проекта № 25 «Тайм-трекер (учёт рабочего времени)». Документ определяет задачи продукта, состав функций, ограничения, пользовательские сценарии, свойства качества и условия проверки результата.

Основание разработки: практическое занятие № 2 по дисциплине «Проектирование и архитектура программного обеспечения». Требования сформированы по результатам интервью с преподавателем-заказчиком.

1.2 Цель, границы и ожидаемый результат

Цель продукта - предоставить пользователю инструмент объективного учёта времени, чтобы он мог сравнивать план с фактом, замечать регулярные отвлечения и корректировать собственное планирование. Программа предназначена для одного владельца ПК и выполняется локально.

В границы проекта включаются создание задач по категориям, управление таймером, учёт пауз с пояснениями, сохранение истории сеансов, фоновый режим с мини-панелью, отчёты по периодам и задачам, круговые диаграммы, история отчётов и рекомендации. Уведомления об истечении планового времени также включены в проект.

Вне границ: регистрация и авторизация, совместная работа нескольких пользователей, общий сервер, облачная синхронизация, мобильное приложение, веб-версия и обязательное шифрование локальных данных. Данные остаются на компьютере пользователя.

1.3 Источники и статус сведений

Таблица 1 - Источники требований и принципы их использования

Код	Источник	Статус
И-1	Интервью с преподавателем	Первичный источник высказываний заказчика.
И-2	Сводка интервью, обсуждение с коллегами, выполняющими аналогичный кейс	Дополняет интервью деталями GUI, мини-панели, круговых диаграмм и истории отчётов.
И-3	«Темы проектов.docx», тема № 25	Задаёт направление и обязательный язык Java.
И-4	«Практическое занятие № 2» и «Лекция № 2» Яковлева М. С.	Определяют структуру SRS, методы выявления требований и UML.
И-5	Рабочие редакции SRS, подготовленные ранее	Черновики: факты перепроверены по И-1-И-4; дополнительные решения не выдаются за слова заказчика.

В документе используется разграничение: «требование заказчика» - прямо зафиксированное ожидание; «проектное решение» - технический способ реализации, выбранный для учебной версии; «допущение» - не подтверждённая интервью детализация, нуждающаяся в согласовании.

1.4 Термины и сокращения

Таблица 2 - Термины и обозначения

Термин	Содержание
SRS	Спецификация требований к программному обеспечению.
GUI	Графический интерфейс приложения.
Задача	Планируемая деятельность с названием, категорией и плановой длительностью.
Сеанс	Отрезок работы над задачей: начало, окончание и интервалы выполнения.

Термин	Содержание
Пауза / прерывание	Перерыв в сеансе, сопровождаемый сохранением причины и длительности.
Активное время	Сумма интервалов работы таймера без пауз.
План / факт	Сопоставление ожидаемой и фактической длительности.
Системный трей	Область уведомлений ОС рядом с часами.
Мини-панель	Компактное окно со статусом задачи и временем, расположенное у нижнего края экрана.
UML	Язык графического моделирования программных систем.
БТ / ПТ / ФТ / НФТ	Бизнес-, пользовательские, функциональные и нефункциональные требования.

2 ОБЩЕЕ ОПИСАНИЕ ПРОДУКТА

2.1 Положение и назначение системы

«Персональный тайм-трекер» - самостоятельное настольное приложение на Java с русскоязычным интерфейсом, предназначенное для учёта времени при самостоятельной работе, учёбе и иных личных задачах. Приложение не зависит от сетевого сервиса и использует локальное хранение.

Заказчик подчёркивает, что продукт должен быть не только секундомером, но и «тренажёром» самоконтроля: пользователь планирует задачу, сам управляет выполнением, фиксирует отвлечения, а программа показывает закономерности и предлагает скорректировать план.

2.2 Типовой цикл использования (Пример)

– Пользователь создаёт категорию «Учёба» и задачу «Подготовка к зачёту», указывает план - 2 часа.
– Нажимает «Старт»: открывается сеанс учёта, таймер измеряет фактическую активную работу.

– При отвлечении нажимает «Пауза» и указывает причину (например, «Телефон»). Причина сохраняется, длительность паузы считается отдельно.
– Нажимает «Продолжить», завершает работу командой «Стоп». Программа фиксирует факт и отклонение от плана.
– В отчёте выбирает неделю и категорию, видит распределение времени, перерывы и рекомендацию, если систематически не укладывается в план.

2.3 Пользователи и внешняя среда

Таблица 3 - Роли и окружение

Элемент	**Описание**
Основной актор	Пользователь-владелец ПК. Он создаёт задачи, управляет сеансами и анализирует статистику.
Внешняя среда	Операционная система Windows предоставляет рабочий стол, системный трей и файловую систему.
Локальное хранилище	Сохраняет задачи, категории, сеансы, паузы и историю отчётов; не является внешним интернет-сервисом.
Подготовка пользователя	Общие навыки работы с настольными программами; администрирование сервера не требуется.

2.4 Предположения и ограничения

Предполагается, что пользователь имеет право создавать локальные файлы в профиле Windows, а системные часы работают корректно. Управлять одной задачей одновременно с другим активным сеансом нельзя: это предотвращает двойной учёт одного и того же времени.

Преподаватель разрешил сторонние библиотеки; однако файл «Темы проектов» требует, чтобы они не подменяли реализацию проекта. Поэтому бизнес-логика учёта времени, обработка пауз, аналитика и генерация рекомендаций реализуются в собственных классах Java.

3 ВЫЯВЛЕНИЕ И АНАЛИЗ ТРЕБОВАНИЙ

3.1 Методы получения требований

Основной метод - интервью в формате «вопрос-ответ» с преподавателем-заказчиком. Метод выбран, поскольку позволяет установить проблему, границы персонального использования и состав аналитики, а также сразу уточнять технические ограничения. Исходная расшифровка помещена в приложение А с сохранением временных меток.

Дополнительные методы - анализ темы и методических материалов, а также User Stories. Анализ документов необходим для проверки технического ограничения Java и полноты SRS; User Stories превращают цели в конкретные действия пользователя. Отдельная анкета или независимое исследование пользователей не проводились и в качестве проведённых работ не заявляются.

3.2 Результаты интервью: подтверждённые положения

Таблица 4 - Фрагменты исходного интервью

Вопрос / тема	**Зафиксированное содержание ответа**
Какую проблему решает проект?	Персональный инструмент против самообмана; человек сам управляет задачами. Главная ценность - глубокая аналитика: план/факт, перерывы, советы, работа в фоне и предупреждения.
Нужна ли авторизация? Кто управляет таймером?	Авторизация не нужна; запуск и остановка осуществляются пользователем вручную.
Какие отчёты и кому доступны?	По виду и конкретной задаче, за период, с показателями, графиками и рекомендациями; доступны владельцу компьютера.
Где хранятся данные?	Локально, без серверного режима.
Формат отчёта и безопасность?	Любой понятный формат; обязательное шифрование не требуется.
Что является рабочим продуктом?	Работающий продукт: задачи по категориям, старт, пауза, продолжение и глубокая аналитика.
Как представить результат?	Документация включает интервью; сдача - до конца октября, только очно с демонстрацией и ответами на вопросы.

Примечание: таблица представляет содержательную выжимку аудио, а не дословный протокол 20 вопросов. Детали «круговые диаграммы», «история отчётов», «мини-панель в правом нижнем углу» получены из отдельной сводки одногруппника и учитываются как дополнительные зафиксированные требования.

3.3 User Stories

- US-01. Как пользователь, я хочу создавать задачи по категориям, чтобы планировать разную деятельность.
- US-02. Как пользователь, я хочу указать плановое время, чтобы позже сравнить его с фактическим.
- US-03. Как пользователь, я хочу запускать, ставить на паузу, продолжать и завершать таймер, чтобы учитывать только реальную работу.
- US-04. Как пользователь, я хочу записывать причину каждой паузы, чтобы анализировать источники отвлечения.
- US-05. Как пользователь, я хочу видеть таймер в компактной панели во время работы с другими окнами.
- US-06. Как пользователь, я хочу получать предупреждения о лимите времени, чтобы вовремя оценивать прогресс.
- US-07. Как пользователь, я хочу построить отчёт за выбранный период и по выбранной задаче, чтобы увидеть закономерности.
- US-08. Как пользователь, я хочу видеть круговую диаграмму по категориям, чтобы понимать распределение времени.
- US-09. Как пользователь, я хочу получать рекомендации на основе статистики, чтобы уточнять планы.
- US-10. Как пользователь, я хочу вернуться к сохранённым отчётам и экспортировать их, чтобы сравнивать результаты.

3.4 Выводы и разрешение неоднозначностей

Исходные пожелания «максимум задач» и «минимальное время отклика» не дают тестируемого числового критерия. Для учебной приёмки ниже введены измеримые целевые показатели, помеченные как проектные предложения. Они не выдаются за значения, названные заказчиком.

Фоновый режим следует разделять на две функции: значок приложения в системном трее и видимую мини-панель на рабочем столе. Системный трей сам по себе не показывает непрерывный отсчёт, поэтому отдельная мини-панель нужна для выполнения сводки интервью.

«Глубокая аналитика» конкретизирована до расчёта план/факт, длительности и причин перерывов, распределения по категориям и рекомендаций по повторяющимся отклонениям. Рекомендации должны быть объяснимыми и воспроизводимыми, без внешних ИИ-сервисов.

4 ТРЕБОВАНИЯ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ

4.1 Бизнес-требования

Таблица 5 - Бизнес-требования

ID	Требование	Источник
БТ-01	Обеспечить объективный персональный учёт времени как основу борьбы с прокрастинацией и самообманом.	И-1
БТ-02	Дать пользователю возможность находить повторяющиеся отклонения планового времени от фактического.	И-1
БТ-03	Снижать непроизводительные перерывы за счёт фиксации их причин и анализа частоты.	И-1, И-2
БТ-04	Предоставлять наглядные отчёты и рекомендации для улучшения планирования.	И-1, И-2

4.2 Пользовательские требования

Таблица 6 - Пользовательские требования

ID	Потребность пользователя
ПТ-01	Создавать и изменять категории и задачи; задавать для задачи плановую длительность.
ПТ-02	Запускать, ставить на паузу, продолжать и завершать учёт времени.
ПТ-03	Указывать причину при каждой паузе и позже видеть её в истории.
ПТ-04	Просматривать списки своих задач и их статусы.
ПТ-05	Наблюдать текущее время и название задачи в компактной панели.
ПТ-06	Продолжать использовать компьютер, пока таймер работает в фоне.
ПТ-07	Получать предупреждение о близком окончании и превышении планового времени.
ПТ-08	Сравнивать плановое и фактическое время по задаче и категории.
ПТ-09	Просматривать количество, длительность и причины прерываний.
ПТ-10	Формировать отчёт за выбранный период с фильтрами по задачам и категориям.
ПТ-11	Видеть круговую диаграмму распределения активного времени по категориям.
ПТ-12	Просматривать рекомендации, историю отчётов и экспортировать отчёт в понятный файл.

4.3 Функциональные требования

В каждой строке указано проверяемое действие системы. Критерии проверки применяются при интеграционных тестах и очной демонстрации.

Задачи и категории

Таблица 7 - Функциональные требования: задачи и категории

ID	Требование	Приёмочный признак
ФТ-01	Система создаёт категорию с уникальным идентификатором и непустым названием.	Категория появляется в списке выбора.
ФТ-02	Система создаёт задачу с непустым названием, категорией и положительной плановой длительностью.	Данные задачи сохраняются и видны в списке.
ФТ-03	Система изменяет название, категорию и план задачи.	Повторное открытие показывает новые значения.

ID	Требование	Приёмочный признак
ФТ-04	Система позволяет удалять или архивировать задачу без повреждения учтённых ранее сеансов.	Исторические показатели не исчезают.
ФТ-05	Система отображает список задач и фильтрует его по категории и статусу.	Список соответствует выбранным условиям.

Таймер и паузы

Таблица 8 - Функциональные требования: таймер и паузы

ID	Требование	Приёмочный признак
ФТ-06	Система запускает не более одного активного сеанса одновременно.	Второй запуск блокируется с пояснением.
ФТ-07	Система ставит активный сеанс на паузу только после ввода непустой причины.	Без причины переход к PAUSED не выполняется.
ФТ-08	Система продолжает выполнение сеанса из состояния PAUSED.	Учёт активного времени возобновляется.
ФТ-09	Система завершает сеанс из RUNNING или PAUSED и фиксирует фактическое активное время.	Создаётся завершённая запись сеанса.
ФТ-10	Система отдельно сохраняет начало и конец каждого интервала паузы, а также её причину.	Данные доступны в журнале и отчёте.
ФТ-11	Система рассчитывает активное время как сумму рабочих интервалов без длительности пауз.	В расчёте отсутствует время прерываний.
ФТ-12	После повторного запуска приложения система восстанавливает незавершённый сеанс без потери сохранённых данных.	Сеанс доступен в состоянии паузы.

Интерфейс и уведомления

Таблица 9 - Функциональные требования: интерфейс и уведомления

ID	Требование	Приёмочный признак
ФТ-13	Система предоставляет русскоязычное основное окно с кнопками управления задачей и таймером.	Основные операции доступны из GUI.
ФТ-14	Система может скрыть основное окно и оставить значок в системном трее.	Управление восстановлением окна доступно через трей.

ID	Требование	Приёмочный признак
ФТ-15	Система показывает мини-панель у правого нижнего края рабочего стола с задачей, временем и состоянием.	Показатели соответствуют текущему сеансу.
ФТ-16	Система показывает уведомление при приближении к плановому лимиту и при его превышении.	В обоих случаях появляется сообщение.
ФТ-17	После остановки таймера система прекращает уведомления об этом сеансе.	Новых уведомлений по нему нет.

Аналитика и отчётность

Таблица 10 - Функциональные требования: аналитика и отчётность

ID	Требование	Приёмочный признак
ФТ-18	Система рассчитывает плановое, фактическое и отклонение времени по задаче.	Показатели рассчитаны из сохранённых сеансов.
ФТ-19	Система вычисляет число пауз, общую длительность и распределение причин по задачам.	Данные совпадают с журналом пауз.
ФТ-20	Система формирует объяснимые рекомендации по систематическому превышению плана и частым отвлечениям.	У совета указаны основание и период расчёта.
ФТ-21	Система строит отчёт за произвольный допустимый период по выбранной задаче и/или категории.	Вывод соответствует применённым фильтрам.
ФТ-22	Система отображает круговую диаграмму распределения активного времени по категориям.	Сумма значений сегментов соответствует итогу.
ФТ-23	Система сохраняет дату создания и параметры каждого сформированного отчёта.	Отчёт виден в истории после перезапуска.
ФТ-24	Система открывает ранее сформированный отчёт из истории.	Показывается сохранённый снимок результата.
ФТ-25	Система экспортирует выбранный отчёт в PDF и CSV.	Файлы открываются и содержат корректные значения.
ФТ-26	Система показывает информативное состояние при отсутствии данных в отчётном периоде.	Нет исключения, деления на ноль и фиктивных графиков.

Хранение и устойчивость

Таблица 11 - Функциональные требования: хранение и устойчивость

ID	Требование	Приёмочный признак
ФТ-27	Система сохраняет категории, задачи, сеансы и паузы в локальном хранилище.	Данные доступны после закрытия и открытия.
ФТ-28	Система корректно обрабатывает недоступность файла хранилища или ошибку записи.	Отображает сообщение и не завершается аварийно.

4.4 Нефункциональные требования

В формулировках ниже значения качества для учебной приёмки являются проектными целями, назначенными на этапе спецификации, а не дословными количественными условиями преподавателя.

Таблица 12 - Нефункциональные требования и способы проверки

ID	Характеристика	Требование	Способ проверки
НФТ-01	Реализация	Java 17 или новее	Проверка версии JDK, успешная сборка.
НФТ-02	Платформа	Основная целевая среда - Windows 10/11.	Запуск и работа интерфейса на Windows.
НФТ-03	Локальность	Данные программы не требуют доступа к сети или серверу.	Запуск и основные операции при отключённом интернете.
НФТ-04	Быстродействие	Целевой ориентир: до 500 мс на 95 % операций отображения/фильтрации 5000 задач на контрольном ПК.	Замер в согласованной среде: 4 ядра, 8 ГБ ОЗУ, SSD.
НФТ-05	Масштаб хранения	Не менее 5000 задач без нарушения целостности и отказа GUI; верхний предел не ограничивать искусственно.	Генерация 5000 записей и проверка операций.
НФТ-06	Актуальность таймера	Обновление показаний интерфейса не реже одного раза в секунду, без блокировки UI.	Наблюдение и проверка отзывчивости.
НФТ-07	Сохранность	Сохранённые данные переживают обычное завершение и перезапуск; незавершённый сеанс возвращается как PAUSED.	Контроль перезапуска в ходе сеанса.

ID	Характеристика	Требование	Способ проверки
НФТ-08	Удобство	Русский интерфейс, ясные названия действий, читаемые сообщения об ошибках.	Демонстрация без консольных команд.
НФТ-09	Воспроизводимость	Советы и числовые отчёты при одинаковых данных и фильтрах детерминированы.	Повторное формирование даёт те же агрегаты.
НФТ-10	Безопасность	Авторизация и шифрование не обязательны; данные хранятся в профиле пользователя.	Проверка отсутствия сетевого обмена.
НФТ-11	Поддерживаемость	Код разделён на модель, сервисы, интерфейс и доступ к данным; ключевые алгоритмы покрываются тестами.	Структура пакетов и JUnit 5.
НФТ-12	Ограничения библиотек	Библиотеки допустимы для GUI/БД/экспорта, но основная предметная логика написана на Java в проекте.	Анализ исходного кода.

4.5 Правила бизнес-логики и корректность расчётов

– Плановое время задаётся в секундах (пользователь видит минуты/часы). Фактическое активное время не включает паузы; длина пауз хранится отдельно.

– Период отчёта задаётся включительно по выбранным календарным датам; внутренний запрос использует полуоткрытый интервал [начало, следующий день).

– Если задача имеет несколько сеансов, факт по задаче равен сумме активного времени всех выбранных сеансов. Один сеанс связывается ровно с одной задачей.

– Суммы по категориям вычисляются из фактических интервалов; круговая диаграмма не строится при нулевом объёме данных.

– При повторяющемся превышении плана приложение предлагает увеличить план или пересмотреть подход к задаче; если данных недостаточно, явно сообщает об этом.
– История отчётов не имеет искусственного ограничения в 100 записей: ранее предложенное число отсутствует в исходном интервью.

5 СПЕЦИФИКАЦИИ ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ

Актор всех пользовательских сценариев - владелец персонального компьютера. Внешняя ОС обеспечивает оконную среду и сохранение файлов, но не является вторым человеком-пользователем.

5.1 UC-01. Создание задачи

Предусловия. Приложение открыто.

Основной поток. Пользователь выбирает категорию, вводит название и план, нажимает «Сохранить»; система проверяет значения и добавляет задачу.

Постусловия. Задача видна в списке и сохранена.

Альтернативный поток. Пустое название либо время ≤ 0: сообщение об ошибке, данные не сохраняются.

Связанные требования. ФТ-01-05.

5.2 UC-02. Выполнение задачи с прерыванием

Предусловия. Существует задача; нет другого активного сеанса.

Основной поток. «Старт» → отсчёт → «Пауза» → ввод причины → «Продолжить» → «Стоп».

Постусловия. Сохранены рабочие интервалы, пауза с причиной и итоговое активное время.

Альтернативный поток. При пустой причине пауза не принимается; повторный «Старт» блокируется.

Связанные требования. ФТ-06-12.

5.3 UC-03. Формирование отчёта и аналитики

Предусловия. Имеются сохранённые сеансы.

Основной поток. Пользователь выбирает период и при необходимости задачу/категорию, запускает построение отчёта.

Постусловия. Показаны план/факт, перерывы, круговая диаграмма и рекомендации.

Альтернативный поток. Нет данных: информативная заглушка без ошибки вычисления.

Связанные требования. ФТ-18-23,26.

5.4 UC-04. Просмотр истории и экспорт

Предусловия. Ранее сохранён хотя бы один отчёт.

Основной поток. Пользователь открывает историю, выбирает отчёт, нажимает «Экспорт», задаёт PDF или CSV и каталог.

Постусловия. Файл сохранён, значения совпадают с отчётом.

Альтернативный поток. Нет прав записи: сообщение, оригинал отчёта сохранён.

Связанные требования. ФТ-23-25.

5.5 UC-05. Работа в фоне и предупреждения

Предусловия. Таймер выполняется.

Основной поток. Пользователь сворачивает окно; мини-панель показывает задачу и время. При приближении к лимиту появляется предупреждение.

Постусловия. Учёт продолжается; основной экран можно восстановить.

Альтернативный поток. Системный трей недоступен: приложение остаётся управляемым из основного окна.

Связанные требования. ФТ-13-17.

6 UML-МОДЕЛИ СИСТЕМЫ

6.1 Диаграмма вариантов использования (Use Case Diagram)

Диаграмма фиксирует границу персонального приложения, единственного актора и доступные ему прецеденты. Обязательное пояснение паузы оформлено отношением «include». Экспорт является необязательным продолжением сценария работы с отчётом.



Рисунок 1 - Диаграмма вариантов использования персонального тайм-трекера

6.2 Диаграмма классов (Class Diagram)

Модель разделяет предметные сущности (Category, Task, WorkSession, PauseRecord), прикладные сервисы (TaskService, TimerService, AnalyticsService, ReportService) и контракт хранения StorageRepository. Это препятствует смешиванию графического интерфейса, вычислений и сохранения данных в одном классе.

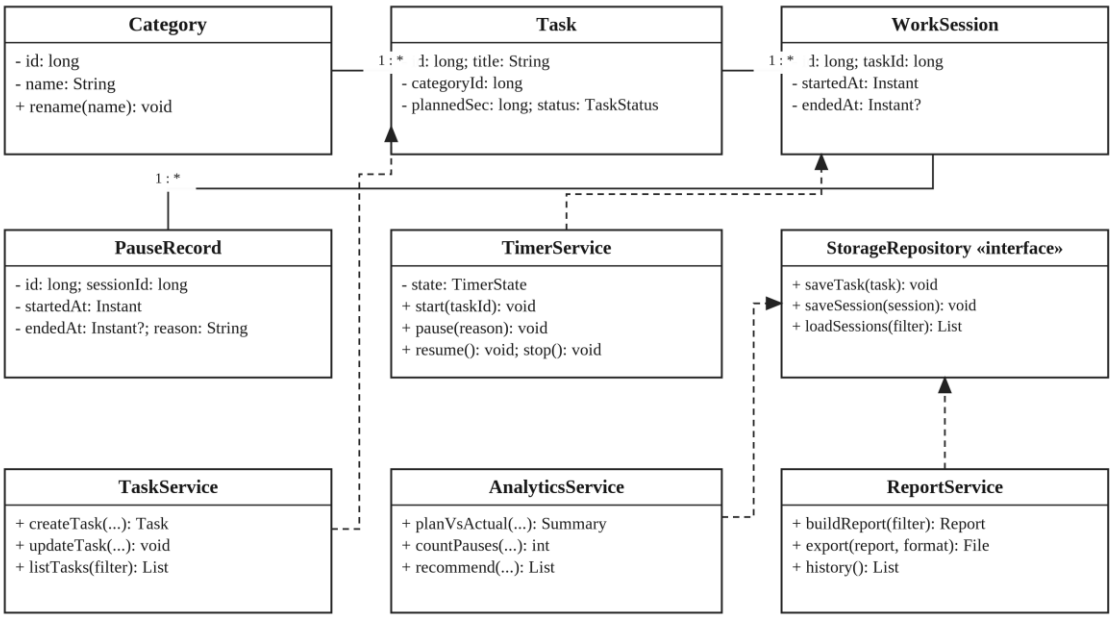


Рисунок 2 - Диаграмма классов и основных отношений

Кардинальности: одной категории соответствует множество задач, одной задаче - множество сеансов, одному сеансу - множество пауз. Репозиторий абстрагирует БД от сервисов; это позволит тестировать предметную логику без реальной базы.

6.3 Диаграмма последовательности: работа с таймером

Сценарий показывает создание задачи, начало работы, паузу с пояснением, продолжение и завершение. Взаимодействие идёт от GUI через сервисы к локальному хранилищу; интерфейс не пишет данные напрямую.

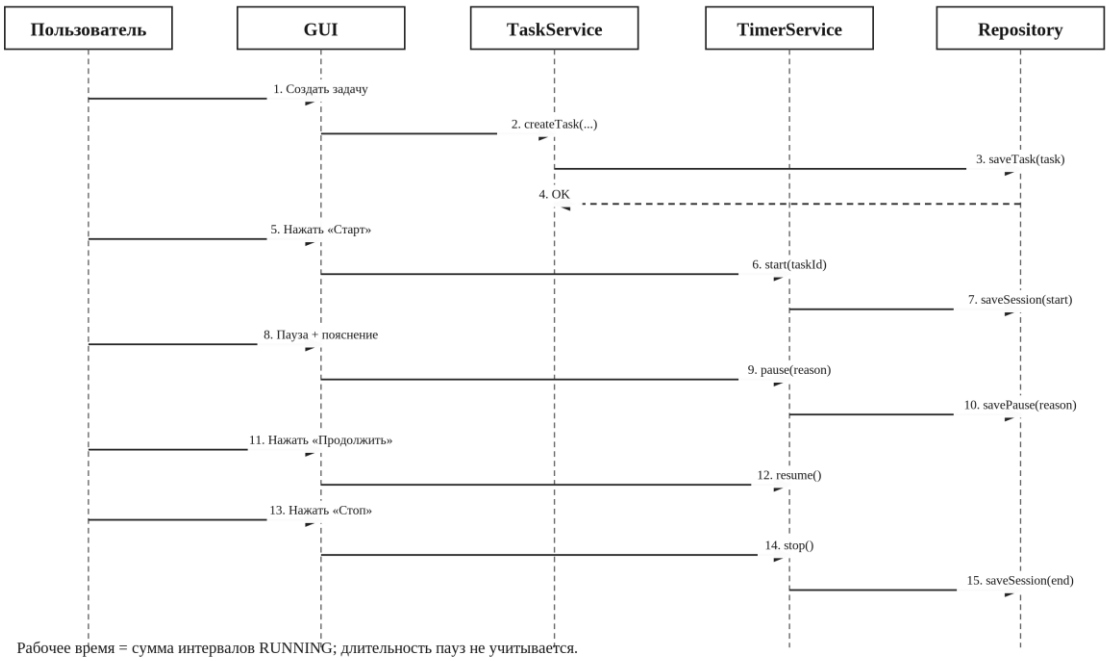


Рисунок 3 - Диаграмма последовательности управления таймером

6.4 Диаграмма последовательности: получение отчёта

При запросе отчёта сервис получает записи сеансов и пауз, передаёт их на расчёт аналитики и сохраняет сформированный снимок; затем пользователь видит результаты и может экспортировать их.

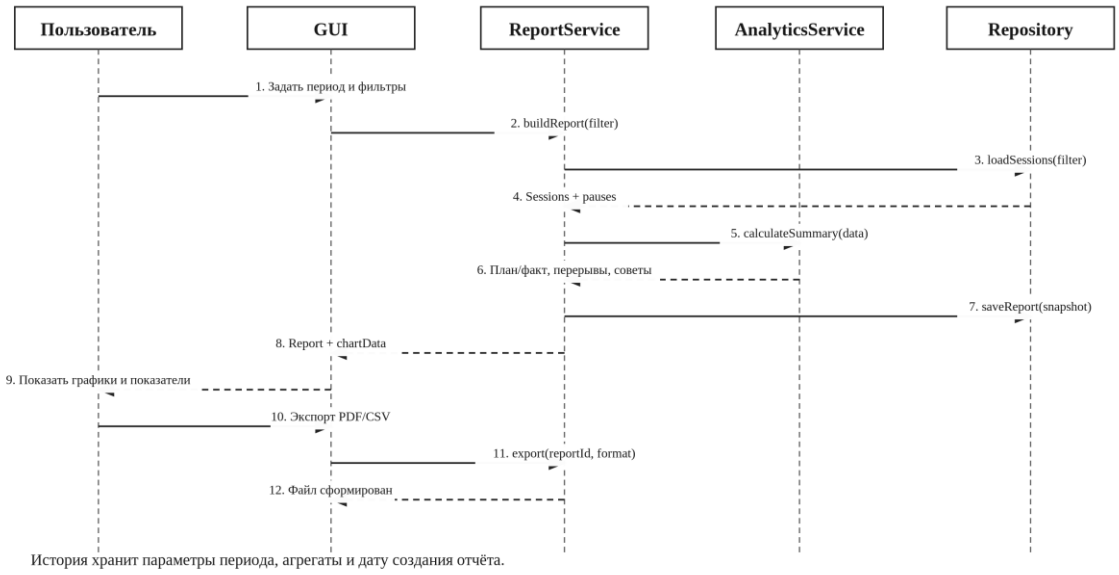
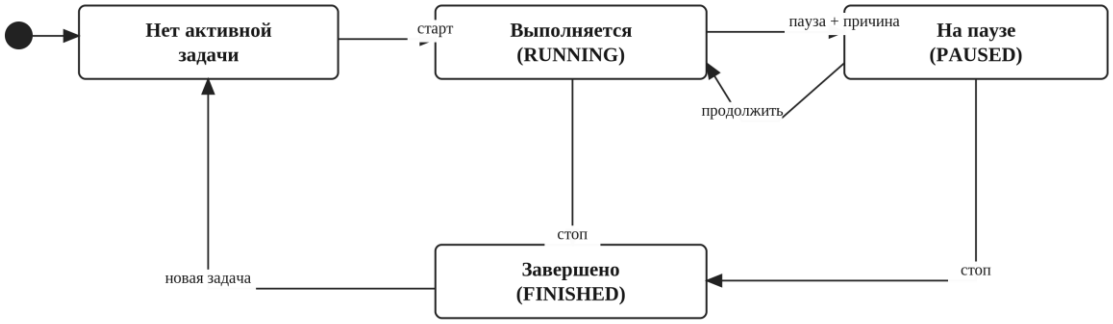


Рисунок 4 - Диаграмма последовательности формирования и экспорта отчёта

6.5 Дополнительная диаграмма состояний таймера

Состояния NOT_STARTED, RUNNING, PAUSED и FINISHED не позволяют вызвать «Продолжить» до паузы либо запустить два независимых активных сеанса одновременно. Переход «Пауза» требует непустой причины.



Системное ограничение: одновременно выполняется не более одной задачи.

Пустая причина паузы не принимается. При перезапуске активный сеанс восстанавливается в состоянии паузы.

Рисунок 5 - Состояния таймера и разрешённые переходы

7 ВЕРИФИКАЦИЯ ТРЕБОВАНИЙ И КРИТЕРИИ ПРИЁМКИ

7.1 Матрица прослеживаемости

Таблица 13 - Связь целей, требований, сценариев и проверки

Цель	**Пользовательские**	**Функциональные**	**Демонстрация**
БТ-01	ПТ-01-04	ФТ-01-12,27	UC-01, UC-02
БТ-02	ПТ-08, ПТ-10	ФТ-18,21,22	UC-03
БТ-03	ПТ-03, ПТ-09	ФТ-07,10,19,20	UC-02, UC-03
БТ-04	ПТ-07-12	ФТ-16,18-26	UC-03, UC-04, UC-05
Устойчивость	ПТ-05, ПТ-06	ФТ-12-17,27-28	Перезапуск и фоновый режим

7.2 Приёмочные проверки функциональности

Таблица 14 - Контрольные приёмочные сценарии

ID	Условие и ожидаемый результат	ФТ
АТ-01	Создать категорию и задачу с планом 20 минут: они появляются после перезапуска.	ФТ-01,02,27
АТ-02	Попытаться создать задачу с пустым названием и нулевым планом: сохранение отклонено.	ФТ-02
АТ-03	Начать задачу, затем начать вторую: система не допускает второй активный сеанс.	ФТ-06
АТ-04	Начать сеанс, приостановить его без причины: пауза отклоняется.	ФТ-07
АТ-05	Ввести причину, пробыть на паузе, продолжить и завершить: пауза не включена в активное время.	ФТ-08-11
АТ-06	Перезапустить программу с незавершённым сеансом: он возвращается в PAUSED.	ФТ-12
АТ-07	Свернуть окно: трей и мини-панель доступны, счётчик продолжает работу.	ФТ-14,15
АТ-08	Создать тестовые сеансы разных категорий и дат: фильтр возвращает только выбранные данные.	ФТ-18,21
АТ-09	Для отчёта сверить вручную сумму активных минут и значения круговой диаграммы.	ФТ-18,22
АТ-10	Сформировать отчёт без данных: появляется объяснение, программа не падает.	ФТ-26
АТ-11	Закрыть и вновь запустить программу: история отчётов доступна.	ФТ-23,24
АТ-12	Экспортировать отчёт в PDF и CSV: файлы открываются, значения корректны.	ФТ-25
АТ-13	Смоделировать отказ записи: приложение отображает понятное сообщение.	ФТ-28

7.3 Контроль аналитики на одном наборе данных

Тестовый пример для воспроизводимой проверки: задача «Учёба» запланирована на 120 минут; пользователь работал 45 минут, затем сделал 15-минутный перерыв с причиной «Телефон», затем работал ещё 105 минут и завершил сеанс. Ожидается: фактическое активное время 150 минут,

суммарная длительность пауз 15 минут, число пауз 1, превышение плана 30 минут, отклонение +25 %.

Для отчёта по двум категориям с активным временем 150 и 50 минут круговая диаграмма должна содержать доли 75 % и 25 %. Если хотя бы одна доля вычислена по полному календарному времени вместе с паузами, проверка считается не пройденной.

7.4 Условия успешного завершения

Результат принимается при успешной демонстрации сценариев AT-01-AT-13, наличии всех трёх обязательных типов UML-диаграмм, корректной сборке проекта на Java и соответствии отчёта реальным возможностям работающего приложения. Количественные цели производительности проверяются отдельно по согласованным тестовым данным.

8 ПРОЕКТНЫЕ РЕШЕНИЯ И ОТКРЫТЫЕ ВОПРОСЫ

8.1 Предварительный технологический выбор

Таблица 15 - Принятые решения учебной версии

Вопрос	**Решение**	**Основание**
Язык и сборка	Java 17 + Maven	Согласуется с ограничением темы и дальнейшими практиками.
Графический интерфейс	Swing (включая SystemTray); мини-панель отдельным компактным окном	Не требует сложной среды развёртывания, подходит для Windows.
Данные	SQLite в локальном файле через JDBC	Удобны запросы по датам/задачам и большое число записей.
Отчёты	Внутренний экран с диаграммой; экспорт в CSV и PDF	Отвечает критерию наглядности и пользовательской сводке.
Рекомендации	Правила на основе статистических порогов, без внешнего ИИ	Объяснимость, повторяемость, собственная реализация.
Модульность	Модели + сервисы + репозиторий + GUI	Подготовка к UML, Maven, JUnit и Docker.

8.2 Проектные допущения, подлежащие подтверждению

Таблица 16 - Параметры, не установленные дословно в интервью

Параметр	**Предложение для разработки**	**Как проверить / уточнить**
Предупреждение о лимите	За 10 % до плановой длительности; дополнительно при превышении.	Показать заказчику пример для 20- и 120-минутных задач.
Частые прерывания	Порог как минимум 3 паузы за один сеанс; совет при превышении.	Утвердить смысл слова «часто».
Советы по планированию	При ≥ 3 завершённых сеансах одной группы сравнивать медианный факт с планом; при отклонении > 20 % рекомендовать пересмотр.	Проверить на примере преподавателя об учёбе.
Восстановление после закрытия	Незавершённый сеанс при следующем запуске возвращается в состояние PAUSED.	Согласовать приёмочный сценарий.
Удаление задач	Архивировать задачу с историей вместо физического удаления связанных сеансов.	Проверить сохранность статистики.
Отклик/объём	Целевой сценарий 5000 задач и 95 % операций до 500 мс на тестовом ПК.	Согласовать тестовую конфигурацию и порог.

Любое изменение этих допущений должно отразиться сразу в требованиях, тестах и UML. В частности, ограничение истории ровно 100 отчётами и обязательная система приоритетов из ранних черновиков не закрепляются: первичное интервью таких требований не содержит.

8.3 Связь с дальнейшими практическими работами

В практическом занятии № 3 проектная модель будет реализована в пакетах Java, а паттерны проектирования выбраны там, где они действительно снижают связанность. В практическом занятии № 4 проверки АТ трансформируются в unit- и интеграционные тесты JUnit 5; в занятии № 5 сборка и тестирование должны выполняться в Docker.

9 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И ПРИМЕНЕНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ

9.1 Архитектурные принципы и уточнение платформенной среды

Настоящая глава дополняет спецификацию требований к программному обеспечению (SRS) проекта «Персональный тайм-трекер на Java». В ней фиксируются архитектурные решения, обоснование выбранных паттернов и способ организации сборки. Выбор выполнен с учётом контекста проекта и рекомендаций лекции № 3: паттерн должен решать конкретную проблему, а его применение не должно создавать избыточную сложность.

Для приложения выбрана слоистая архитектура. Представление (MainWindow и MiniTimerWindow) отвечает за взаимодействие с пользователем. Прикладной слой содержит сценарии работы с задачами и таймером. Доменная модель описывает категории, задачи, рабочие сеансы и паузы, а интерфейсы репозиториев отделяют правила приложения от конкретного способа хранения. Инфраструктурный слой содержит реализации SQLite и адаптер интеграции с рабочим столом.

Уточнение по замечанию преподавателя. В первоначальной редакции SRS Windows была указана как основная целевая операционная система. Это решение не следует считать окончательным ограничением. В дальнейшей разработке целевая ОС не фиксируется заранее: совместимость с Windows, Linux и macOS проверяется отдельно. Основная логика не должна зависеть от наличия системного трея; если конкретная графическая среда его не поддерживает, используется резервная реализация NoOpDesktopIntegration, а основное окно остаётся доступным.



Рисунок 6 — Слоистая архитектура и границы компонентов приложения

9.2 Выбор и обоснование паттернов проектирования

Паттерны отбирались по наличию конкретной задачи в проекте, а не по количеству. В результате выбраны три поведенческих паттерна, один структурный и один порождающий. Такой набор покрывает управление состояниями таймера, рассылку обновлений, заменяемые алгоритмы аналитики, интеграцию с API рабочего стола и создание форматов экспорта.

Паттерн и категория	Проблема проекта	Классы реализации
State — поведенческий	Набор допустимых действий зависит от состояния таймера.	TimerService, TimerState, IdleState, RunningState, PausedState, FinishedState.
Observer — поведенческий	Несколько элементов интерфейса должны получать одинаковые обновления таймера.	TimerService, TimerObserver, TimerSnapshot, MainWindow, MiniTimerWindow.
Strategy — поведенческий	Правила выдачи рекомендаций могут изменяться и комбинироваться.	RecommendationService, RecommendationStrategy, OvertimeRecommendationStrategy, FrequentPauseRecommendationStrategy.
Adapter — структурный	Прикладной код не должен зависеть от конкретного API системного трея.	DesktopIntegration, AwtDesktopIntegrationAdapter, NoOpDesktopIntegration.
Factory Method — порождающий	Создание PDF- и CSV-экспортёров должно выбираться по формату отчёта.	ReportExporterFactory, CsvReportExporterFactory, PdfReportExporterFactory, ReportExporter.

9.3.1 Паттерн State (Состояние)

Контекст — TimerService. Интерфейс TimerState задаёт операции start, pause, resume и stop. Конкретные состояния IdleState, RunningState, PausedState и FinishedState определяют, какие действия допустимы в данный момент. TimerService делегирует операцию текущему объекту состояния, а после успешного действия заменяет его следующим состоянием.

Пример взаимодействия: после запуска задачи TimerService переходит в RunningState. Вызов pause(reason) передаётся текущему состоянию; RunningState разрешает операцию и инициирует сохранение причины паузы, после чего устанавливается PausedState. Вызов resume() в PausedState закрывает запись о паузе и возвращает таймер в RunningState. Попытка выполнить недопустимую операцию приводит к IllegalStateException. Это исключает разрастание одного метода множеством условных операторов.

9.3.2 Паттерн Observer (Наблюдатель)

TimerService выступает источником событий, а MainWindow и MiniTimerWindow — подписчиками. Контракт TimerObserver содержит метод onTimerUpdated(TimerSnapshot). При старте, паузе, продолжении,

остановке и периодическом обновлении TimerService формирует неизменяемый снимок состояния и передаёт его всем подписчикам.

Список наблюдателей хранится в CopyOnWriteArrayList, поэтому изменение подписок не требует ручного управления синхронизацией при обходе коллекции. Исключение, возникшее в одном представлении, не должно прекращать уведомление остальных. Графический интерфейс переносит обновление компонентов на поток Swing через SwingUtilities.invokeLater. Благодаря этому таймер не зависит от количества окон, которые отображают его состояние.

9.3.3 Паттерн Strategy (Стратегия)

RecommendationService хранит список объектов RecommendationStrategy и последовательно вызывает evaluate(AnalyticsInput). Каждая стратегия анализирует свой критерий и возвращает Optional<Recommendation>: отсутствие рекомендации означает, что соответствующее условие не выполнено.

В текущем прототипе реализованы OvertimeRecommendationStrategy (сравнение фактической и плановой длительности с заданным порогом) и FrequentPauseRecommendationStrategy (анализ количества пауз). Новое правило можно добавить отдельным классом, реализующим RecommendationStrategy, не изменяя алгоритм RecommendationService. Числовые пороги остаются проектными параметрами и не выдаются за подтверждённые ответы заказчика.

Преимущество решения — независимое развитие правил аналитики и возможность проверять каждую стратегию отдельно. Ограничение — набор стратегий должен быть настроен при создании сервиса; пользовательское редактирование правил не входит в текущую промежуточную версию.

9.3.4 Паттерн Adapter (Адаптер)

Целевой интерфейс DesktopIntegration изолирует прикладную часть от API рабочего стола. AwtDesktopIntegrationAdapter реализует его средствами

AWT: проверяет доступность SystemTray, добавляет TrayIcon, отображает уведомления и предоставляет команды открытия или завершения приложения. DesktopIntegrationFactory выбирает адаптер только при наличии графической среды и поддержке системного трея.

Если системный трей недоступен или среда работает в headless-режиме, выбирается NoOpDesktopIntegration. Резервная реализация не выполняет платформенных действий, но сохраняет единый контракт. Это позволяет не привязывать основную логику к Windows и постепенно проверять поведение на других ОС.

9.3.5 Паттерн Factory Method (Фабричный метод)

ReportExporterFactory является абстрактным создателем и объявляет фабричный метод createExporter(), возвращающий интерфейс ReportExporter. Конкретные создатели CsvReportExporterFactory и PdfReportExporterFactory возвращают CsvReportExporter и PdfReportExporter соответственно. ReportService выбирает конкретную фабрику по значению ExportFormat и вызывает общий метод export().

Клиентский код зависит от интерфейса ReportExporter, а не от конкретной библиотеки или формата. Добавление нового формата потребует нового экспортёра и конкретной фабрики; существующий интерфейс сервиса при этом сохраняется.



Рисунок 7 — Участники пяти выбранных паттернов проектирования

10 СИСТЕМА СБОРКИ И ОРГАНИЗАЦИЯ JAVA-ПРОЕКТА

10.1 Выбор Maven

Для сборки выбран Apache Maven. В лекции № 4 Maven описан как декларативный инструмент: в pom.xml задаются координаты проекта, версия Java, зависимости и плагины, а выполнение организуется через стандартный жизненный цикл. Для учебного проекта это обеспечивает предсказуемую структуру и воспроизводимую сборку.

Система	Особенность	Оценка для проекта
Ant	Императивное описание шагов в build.xml; зависимости и classpath часто приходится настраивать вручную.	Не выбран: для текущего проекта потребовал бы больше ручной конфигурации.
Maven	Декларативный pom.xml, стандартные каталоги, жизненный цикл и управление транзитивными зависимостями.	Выбран: соответствует методичке и упрощает сборку, подключение библиотек и упаковку JAR.
Gradle	Гибкий DSL на Groovy/Kotlin, граф задач и развитая инкрементальная сборка.	Возможен, но для проекта не даёт критичного преимущества перед Maven.

10.2 Координаты и конфигурация pom.xml

Параметр	Значение
groupId	ru.mirea.khusyainov
artifactId	personal-time-tracker
version	1.0.0
packaging	jar
Версия компиляции	Java 17 (maven.compiler.release=17)
Главный класс	ru.mirea.khusyainov.timetracker.App
Итоговый файл	target/personal-time-tracker.jar

В pom.xml настроены maven-compiler-plugin 3.13.0, maven-surefire-plugin 3.2.5 и maven-shade-plugin 3.5.3. Последний формирует исполняемый JAR с runtime-зависимостями и записывает главный класс в манифест. Используется ServicesResourceTransformer для сохранения записей SPI подключённых библиотек.

10.3 Зависимости, структура и проверка сборки

Зависимость	Версия / область	Назначение
sqlite-jdbc	3.46.1.0, runtime	Локальная база SQLite через JDBC.
OpenPDF	1.3.39, runtime	Формирование PDF-отчётов.
JUnit Jupiter	5.10.2, test	Подготовленная тестовая зависимость для практической работы № 4; в runtime JAR не включается.

Исходный код содержит 44 Java-файла и разделён на пакеты domain.model, domain.repository, application.task, application.timer, application.analytics, application.report, infrastructure.persistence, infrastructure.platform и presentation. Ресурсы находятся в src/main/resources; в том числе включён шрифт DejaVu Sans с лицензией для одинакового отображения кириллицы при экспорте PDF на разных ОС. Каталог target создаётся Maven и не хранится в Git.

README.md содержит назначение приложения, используемые библиотеки, структуру каталогов, предварительные условия, команду сборки и пример запуска. Для сборки используется команда:

```
mvn clean package
```

После успешной сборки исполняемый файл запускается командой:

```
java -jar target/personal-time-tracker.jar
```

Для автоматизированной проверки собранного JAR без открытия графического окна предусмотрен диагностический режим:

```
java -jar target/personal-time-tracker.jar --self-check
```

Самопроверка создаёт временную базу SQLite, проверяет сохранение сеанса и причины паузы, выполнение стратегий рекомендаций и создание CSV/PDF-файлов. В GitHub Actions на коммите fda8319 успешно завершились сборка и самопроверка на Ubuntu, Windows и macOS: mvn clean package и запуск JAR в режиме --self-check. Это подтверждает переносимость сборки и базовых интеграций; фактическое поведение GUI и системного трея на каждой ОС дополнительно проверяется в соответствующей графической среде.

Список использованных источников

1. Яковлев М. С. Лекция № 3 «Паттерны проектирования». — МИРЭА — Российский технологический университет, 2026.
2. Яковлев М. С. Лекция № 4 «Системы сборки». — МИРЭА — Российский технологический университет, 2026.
3. Практическое занятие № 3 «Паттерны проектирования. Системы сборки» по дисциплине «Проектирование и архитектура программного обеспечения». — 2026.
4. Хусяинов Э. Р. Спецификация требований к программному обеспечению «Персональный тайм-трекер на Java». Практическое занятие № 2. — 2026.

ЗАКЛЮЧЕНИЕ

В рамках практического занятия № 2 сформирована спецификация требований к персональному тайм-трекеру. В документе определены назначение и границы системы, роли пользователя, источники требований, функциональные и нефункциональные требования, пользовательские сценарии, UML-модели и критерии приёмки. В приложение включён протокол исходного интервью с заказчиком.

В практическом занятии № 3 определена архитектура приложения и обосновано применение пяти паттернов проектирования: State, Observer, Strategy, Adapter и Factory Method. Для каждого паттерна приведены конкретные участники и их взаимодействие в Java-проекте. Отдельно уточнено, что Windows не является единственной заранее установленной целевой операционной системой: поддержка Windows, Linux и macOS должна проверяться в соответствующих графических средах.

Для организации проекта выбрана Apache Maven. Исходный код разделён на логические пакеты, в pom.xml определены координаты проекта, версия Java, зависимости и плагины, а maven-shade-plugin настроен на создание исполняемого JAR. В репозитории размещены исходный код, README.md, редактируемые диаграммы draw.io и материалы практических работ.

Текущая версия является промежуточной реализацией, предназначенной для демонстрации архитектуры и сборки. Полная аналитика, фильтрация отчётов, история отчётов, восстановление незавершённых сеансов и расширенный набор пользовательских настроек остаются задачами следующих этапов разработки. Практическая работа № 4 должна дополнить проект модульными и интеграционными тестами JUnit 5, а практическая работа № 5 — контейнеризацией и запуском тестов в Docker.

Практические занятия № 2 и № 3 связаны единым проектом: требования заказчика определяют поведение приложения, архитектура задаёт границы компонентов, паттерны локализуют изменяемые части, а Maven обеспечивает воспроизводимую сборку.

Таймтрекер таймтрекер? Да. Вот. Вот я на все вопросы ему уже ответил. Если хочешь, ты фиксировал как-то интервью, вот можешь у него поинтересоваться.

Спикер 1, 00:01:16. Вот. Ну, соответственно, то есть перекрёстными интервью какие-то вопросы ты не задашь, а у него уже есть. Хорошо, я понял. Соответственно, глубокой аналитикой, которая показывает и даёт. Советы и, так сказать, показатели того, что меньше ты отвлекаешься, может, какие-то классы, задач нужно времени больше уделять.

Спикер 1, 00:01:39. Ну, типа, я себе ставлю задачу учёбы каждый день на 2 ч и каждый день её в итоге делаю три-четыре часа. Мне, соответственно, при проведении аналитики программа говорит: «Дружочек-пирожочек, у тебя задачи по учёбе ты не выполнила за поставленное время ноль раз за неделю. Давай мы тебе время увеличим». Или там слишком много отвлекаешься.

Спикер 1, 00:02:00. Плюс она должна работать в фоновом режиме с показателями и аллертами о том, когда там времени мало осталось, либо там.

Спикер 2, 00:02:08. А нужна ли авторизация пользователя?

Спикер 1, 00:02:10. Нет, это персональная программа.

Спикер 2, 00:02:12. То есть пользователь вручную сам включает таймер, сам выключает.

Спикер 1, 00:02:17. Да.

Спикер 2, 00:02:21. Какие отчёты нужны и кому они доступны? Это вот как раз глубокая по задачам,

Спикер 1, 00:02:27. По типу задач, по конкретной задаче за период, соответственно, с показателями, с графиками, с диаграммами. ...С советами, с показателями каких-то проблем. Они доступны тому чей комп.

Спикер 2, 00:02:43. Пару слов про технические рамки. Где хранить данные лучше? Это файлы, файлы?

Спикер 1, 00:02:49. Либо файлы серверная, локальная. Нет, нет, всё персонально на Dexpo.

Спикер 2, 00:02:55. Ну, отчёт — это скорее всего PDF,

Спикер 1, 00:02:56. CS в любом формате. Главное, чтобы он был наглядный, потому что для обычных людей.

Спикер 2, 00:03:03. Требования к безопасности должны были быть никаких. Вообще-то, да, это локально работает.

Спикер 1, 00:03:08. Даже шифрование никакого делать не надо.

Спикер 2, 00:03:11. А минимально рабочий продукт здесь что будет?

Спикер 1, 00:03:13. А здесь нет понятия MP. У вас в этом семестре рабочий продукт. Угу. Это возможность создавать задачи по категориям, запускать их, ставить на паузу, продолжать выполнение и получать глубокую аналитику моей деятельности.

Спикер 2, 00:03:29. Я понял. Ну, в принципе, тут всё логически понятно. Вопрос вот про документацию. Мы интервью включаем в документацию, пишем вопросы-ответы. Да и до конца октября сдаём. Там до тридцатого.

Спикер 1, 00:03:42. До конца октября сдаём. Всё. Она сдаётся только очным. Естественно. То есть, если вы не придёте на пару и не покажете и не ответите на мои вопросы, то всё понятно. Ничего не получится. Спасибо.